

RuleChef: Grounding LLM Task Knowledge in Human-Editable Rules

Ádám Kovács¹, Nadia Verdha², Gábor Recski^{1,2}

¹KR Labs, ²TU Wien

Correspondence: kovacs@krlabs.eu

Abstract

We present RuleChef, a framework that uses large language models (LLMs) to generate executable rules for NLP tasks such as text classification, Named Entity Recognition (NER), or relation extraction. Rules are generated based on a task description and a set of labeled examples, then they are iteratively improved based both on additional examples and on human feedback over existing rules. RuleChef can also be used to bootstrap rules using the observed input-output pairs from any existing model for a given task. LLMs are used only at learning time, synthesizing rules and iteratively patching them based on failures measured on a held-out split. The result of this process is a fast, deterministic, and inspectable rule system. Preliminary evaluation is performed on both classification and NER tasks. We release RuleChef as open-source software under an Apache 2.0 license¹.

1 Introduction

Early NLP systems were predominantly rule-based: hand-crafted patterns and template grammars provided transparent, deterministic models that could be inspected, debugged, and explained to end users (Chiticariu et al., 2013; Valenzuela-Escárcega et al., 2020; Kovács et al., 2022). The shift to feature-based classifiers and then to pretrained transformers (Devlin et al., 2019) and large language models (LLMs) brought substantial gains in coverage and flexibility, but also moved decision logic into latent parameters that are difficult to audit or edit directly (Lipton, 2018). For applications in sensitive and highly regulated domains such as medical, legal, or financial NLP, a prediction that cannot be traced to an explicit pattern cannot be certified. For many real-world NLP tasks, especially those with recurring lexical or structural patterns,

explicit rules remain attractive: cheap to run, deterministic, easy to version, and straightforward to inspect. The main problem of rules is not their accuracy; it is that writing and maintaining them by hand is labor-intensive and requires substantial domain expertise (Chiticariu et al., 2013; Shnarch et al., 2017; Kovács et al., 2022).

RuleChef addresses this gap by using LLMs for *rule learning*: instead of using neural models at inference time, their role is to translate supervision into executable symbolic rules. Given a task specification and supervision signal S , RuleChef synthesizes a rule set $R = \{r_1, \dots, r_k\}$, evaluates it on a held-out development split, clusters the failures, and prompts the LLM to improve the rules. A patch is accepted only if it improves quality on the held-out split, and each surviving rule records the precision it reached there. Rules can also be reviewed by human domain experts, and their feedback as well as explicit corrections can also be used to prompt LLMs to make updates.

The main contributions of this paper are the following:

- A system for synthesizing executable rules from task supervision (examples, corrections, free-text feedback, or observed model behavior), separating learning-time LLM use from inference-time deterministic execution.
- A refinement loop that iteratively improves a rule system, measures the impact of changes on held-out data and resolves conflicts between rules by their held-out precision.
- Evaluation on two tasks, comparing learned rules against prompting the same LLM and against a dedicated neural extractor, with an ablation study isolating the pipeline’s contribution over one-shot rule prompting.
- Additional experiments on the human-in-the-loop repair process and the unsupervised ob-

¹<https://github.com/KRLabsOrg/rulechef>

servation mode that learns rules from the behavior of an external model.

The remainder of this paper is structured as follows. Section 2 reviews related work on rule synthesis, weak supervision, and LLM-based program generation. Section 3 describes the RuleChef framework. Section 4 presents our experimental setup. Section 5 reports preliminary evaluation results on a simple intent classification dataset and a more challenging NER task. Section 6 discusses key findings and presents opportunities for future work.

2 Related Work

In this section we review prior work related to RuleChef across four areas: automatic rule and regex synthesis, interactive rule systems, weak supervision, and LLM-based code and rule generation. Automatic regex synthesis from examples has been studied with evolutionary methods (Bartoli et al., 2016, 2018), neural sequence-to-sequence models (Locascio et al., 2016; Zhong et al., 2018), and systems that combine natural language descriptions with positive and negative examples (Chen et al., 2020; Li et al., 2021). These approaches produce a single rule or small rule set from examples, but do not include iterative validation or a human-in-the-loop component.

Interactive rule systems such as HEIDL (Sen et al., 2019), GrASP (Shnarch et al., 2017; Lertvitayakumjorn et al., 2022), Odinson (Valenzuela-Escárcega et al., 2020), and POTATO (Kovács et al., 2022) help users build patterns over texts as well as over graph-based representations of semantics and syntax (e.g. Abstract Meaning Representations (Banarescu et al., 2013) and Universal Dependencies (Nivre et al., 2020)), but still require substantial manual authoring. RuleChef reduces this manual effort by creating an interface between various sources of supervision (examples, corrections, feedback on rules) and the LLM performing updates to the rule system.

Weak supervision systems such as Snorkel (Ratner et al., 2017) and Snuba (Varma and Ré, 2018) use labeling functions to create training labels, and recent work also uses LLM prompts as labeling functions (Smith et al., 2024; Yu and Bach, 2023). Unlike these systems, RuleChef makes the rules the final model rather than a source of noisy labels for a downstream classifier. LLMs have also been used to synthesize executable programs from examples,

including Evaporate-Code+ (Arora et al., 2023) and Hypothesis Search (Wang et al., 2024), broader surveys discuss LLM-based rule and hypothesis generation (Sivasothy et al., 2024; He and Chen, 2025). RuleChef can also be viewed as the symbolic alternative for LLM knowledge distillation (West et al., 2022; Zhou et al., 2024; Hsieh et al., 2023) that maximizes explainability and minimizes inference cost. For the two tasks evaluated here, the relevant neural baselines are GLiNER (Zaratiana et al., 2024) and its schema-driven successor GLiNER2 (Zaratiana et al., 2025) for zero-shot extraction of text spans, and dual sentence encoders for intent detection (Casanueva et al., 2020; Zhang et al., 2021).

3 The RuleChef Framework

In this section we describe the main functionalities of the RuleChef framework, including task definition, rule synthesis, the refinement loop, conflict resolution, and the observation mode that allows autonomous operation in the presence of a pre-existing model. A high-level overview is presented in Figure 1. We draw the examples in this section from the Text Anonymization Benchmark dataset (TAB, see Section 4), our main testbed. The prompt templates corresponding to the various rule learning strategies described here are presented in Appendix A.

3.1 Rule Synthesis

We structure the synthesis prompt in four sections: (1) the task definition with input/output schemas, (2) sampled training examples and corrections, (3) data evidence with pattern suggestions, and (4) format instructions with response schema. For multi-class tasks, we generate one prompt per label with positive examples and counter-examples from other classes, preventing cross-class interference.

The task definition specifies input/output schemas and one of four task types: *Classification*, *NER*, *Extraction* (untyped spans), or *Transformation* (structured output). Rules can be requested in one of three formats, including regular expressions (regexes), spaCy rules that rely on the output of part-of-speech tagging and syntactic parsing, and arbitrary Python code. While each of these three formats is supported by RuleChef, the examples and evaluation in this paper focus only on regex patterns. Each generated rule is stored along with metadata that specifies its priority and measures

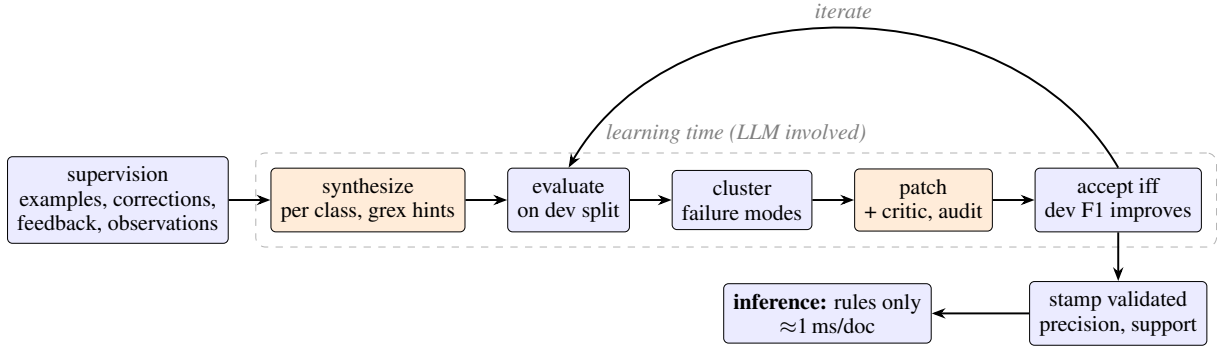


Figure 1: The RuleChef pipeline. Orange components call the LLM; blue components are deterministic. The LLM proposes rules and patches, but acceptance is decided by quality measured on a held-out development split, and each rule records its precision there. At inference time only the rules run.

its precision and match-count over the held-out development set.

By default the data evidence provided to the LLM consists of labeled examples, but RuleChef can also augment this with regex suggestions obtained from grex (Stahl, 2019), a regex generator that derives structural patterns from example strings. Such patterns serve as hints, not constraints, helping the LLM identify structural regularities without overfitting. We validate each generated rule before acceptance: regex patterns must compile, output templates must match the task schema, and patterns that match arbitrary text are rejected by probing them against generic strings.

3.2 Refinement

After initial synthesis, a refinement loop evaluates the current rules, identifies failures, and generates *patch rules* targeting missed or misclassified inputs. Data is split into training and development portions (explicit user corrections always stay in train, since they are the highest-value signal for patching), and the synthesis step for rule patching can only access training data, the dev split is used to decide whether a newly generated rule is accepted. A patch is kept only if held-out F1 does not degrade or precision improves. This filter prevents memorization: as we show in Section 5.2, the same loop without holdout acceptance drifts toward patterns that fit the failures it was shown rather than ones that generalize.

With hundreds of training documents, the failures of an intermediate rule set number in the thousands, far more than fit in a patch prompt. RuleChef clusters failures by their signature. For NER-type tasks, failure modes include *missed span*, *spurious span*, and *wrong type*. The LLM is presented with the full distribution of failure modes plus a

Rule: "case_and_echr_numbers", P=86% on dev
failure_mode: CODE missed_span (84 cases),
e.g. "nos. 6210/73 and 6877/75"

Figure 2: Example input rule for the refinement step. The rule is referred to by its name, followed by its dev-set precision, failure type, and number of matches

sample of failed instances from each cluster. The patch prompt also carries the current rule set with per-rule metrics and all accumulated feedback, an example is shown in Figure 2. The LLM can modify existing rules, add new ones, or delete overly broad rules when providing narrower replacements.

For streaming or batch-wise data, RuleChef can skip the initial synthesis step and update an existing ruleset directly: we commit new examples, corrections, and feedback, evaluate the current rules, and use the resulting failures to drive patch synthesis, preserving the rest of the ruleset. This is the mechanism behind the human-feedback experiment described in Section 5.3.

Agentic coordination. The loop can be driven by a fixed schedule or by an LLM that reasons about the current state. The *simple coordinator* runs a fixed number of iterations; the *agentic coordinator* instead reads the per-class metrics after each iteration to steer the next patch, runs a periodic *critic* that adds rule-level feedback through the same channel as a human, and runs a periodic *audit* that merges redundant rules or removes ineffective ones, reverting any change that lowers measured quality. Appendix A shows the complete prompt templates for each agent, Section 5.2 measures their effect.

3.3 Conflict Resolution and Pruning

When learning concludes, we run each rule alone on the development split and record its precision there along with the number of matches. When two rules overlap or disagree on a span, the executor keeps the match from the higher-priority rule, breaking ties based on dev-set precision. Low-support estimates are discounted by a Wilson lower bound (Wilson, 1927), so a rule right twice out of twice does not outrank one right 95 times out of 100. A leave-one-out pass measures each rule’s marginal contribution to ensemble F1 and drops rules whose removal does not hurt it.

Measuring precision on data the rule was not tuned on also separates good rules from memorized ones. On the TAB data, the rule `case_and_echr_numbers`, corresponding to the regex `(?:no\.?\s*)(\d{4,6}/\d{2,4})`, achieves 0.86 precision with 22 true positive matches on the development set. The overly generic rule `QUANTITY`, by contrast, matched only two development spans at precision 1.00, which the Wilson bound rightly discounts: the rule also matched fraction-shaped case numbers such as *1432/03* (5.7 F1 on test) before the feedback repair step (see Section 5.3). Rules that memorized training lexicons show up the same way, with low development-set precision (see Section 5.4).

Observation mode. For settings where an LLM already handles production traffic, RuleChef can use the model’s behavior as a supervision source rather than requiring labels upfront. Each LLM call is treated as a training example, and RuleChef periodically synthesizes rules from the accumulated observations using the standard pipeline. Matched queries are then routed to rules instead of the LLM. A deployment can delegate only to rules whose measured precision exceeds a threshold, leaving the rest to the LLM. This mode also supports *task discovery*, where RuleChef prompts an LLM to discover the full task specification based on the input-output examples. This mode allows the system to generate an initial rule system based on an external black box model without human input.

4 Experimental Setup

We provide preliminary evaluation of the RuleChef methodology introduced in this paper via experiments on two datasets that represent two of the most common NLP tasks, Named Entity Recognition and text classification. In this section we

describe the datasets and the experimental setup, results follow in Section 5.

4.1 TAB: Anonymization of Court Decisions

Our primary experiment uses the Text Anonymization Benchmark (TAB; Pilán et al., 2022), a corpus of 1,268 decisions of the European Court of Human Rights, with human annotation marking all text spans that represent personal information. We use TAB’s eight official entity types unchanged: `PERSON`, `CODE` (case numbers, phone numbers, license plates, etc.), `DATETIME`, `QUANTITY`, `ORG`, `LOC`, `DEM` (demographic attributes such as nationality or profession), and `MISC`. For analysis we additionally group them by what governs their surface form: *format types* (`CODE`, `DATETIME`, `QUANTITY`), whose mentions follow formal patterns, and *semantic types* (`PERSON`, `ORG`, `LOC`, `DEM`, `MISC`), defined by meaning rather than form.

Since full documents are too long to fit into the context window of LLMs for rule learning, we segment them into chunks of at most 600 characters. For the experiments described here we generate a sample of 1,000 training and 600 test chunks. Rule systems generated by RuleChef are evaluated on this test set as well as on the official test split, which contains 127 full, unchunked documents. RuleChef uses Kimi-K2.6 as the rule-writing LLM with the agentic coordinator, three refinement iterations, and a 20% development holdout; at inference time only the learned rules run. Our main baseline for comparison is GLiNER2 (Zaratiāna et al., 2025), a 205M-parameter schema-driven extractor.

4.2 Banking77: Intent Classification and Observation Mode

The text classification experiments use Banking77 (Casanueva et al., 2020), a dataset of over 13k customer service queries classified by user intent into 77 categories that include classes that are easily detected via keywords (such as `exchange_rate`) but also more challenging ones such as `beneficiary_not_allowed`. A set of 200 examples spanning five intent classes is used as the held-out test set in our experiments. Given the relative simplicity of this task, our baseline for comparison is based on directly prompting the same LLM that we use for rule generation (Kimi-K2-Instruct). This approach achieves over 98% accuracy on this test set and is also used to evaluate RuleChef in Observation mode, where system-generated labels provide the only source of supervision for rule gen-

eration. At each iteration we measure rule coverage, precision, and the fraction of subsequent LLM calls replaced. As context, fine-tuned dual-encoder and contrastive models reach 86–87% accuracy on the full 77-class task with 10 shots per class (Zhang et al., 2021).

5 Results

In this section we report results on the two tasks introduced in Section 4. Sections 5.1–5.3 evaluate on TAB, comparing RuleChef-generated rules against published baselines and measuring the impact of the feedback-repair process. Section 5.4 provides qualitative analysis of the rules learned on the TAB dataset. Section 5.5 uses the Banking77 dataset, measures performance against few-shot classification approaches, and evaluates RuleChef’s observation mode.

5.1 Rule system performance

System	FORMAT			SEMANTIC			ms/doc
	P	R	F1	P	R	F1	
RuleChef (rules only)	89.1	70.5	78.7	75.7	34.9	47.8	0.6
LLM prompting	65.1	87.9	74.8	54.6	60.5	57.4	≈1500
GLiNER2 (labels)	66.5	78.0	71.8	31.7	46.2	37.6	190
GLiNER2 (schema)	69.6	79.9	74.4	34.9	49.0	40.8	190

Table 1: Results on the 600 test chunks of the TAB dataset. The *Format* group includes the entity types Code, Datetime and Quantity, the *Semantic* group includes Per, Org, Loc, Dem, and Misc.

Table 1 compares the performance of learned rules against direct LLM prompting and two GLiNER2 baselines. We observe that learned rules are superior to all other systems on classes that can typically be detected based on the surface form of the entities, while on the more challenging types their overall performance remains below that of direct LLM prompting, but they still outperform the GLiNER baselines and also achieve the highest precision among all systems. Higher precision at the cost of lower recall is the intended behavior, since RuleChef is designed to generate individual rules with high precision, ensuring that incrementally growing a rule system by adding additional patterns can eventually lead to a system with high precision and high recall. A qualitative analysis of learned rules is provided in Section 5.4. We also measure inference latency for each system to quantify the obvious fact that using regex-based rules means having virtually zero inference costs.

Meanwhile, the rule learning process involved less than 20 LLM calls and took approx. 12 minutes.

System	P	R _{all}	R _{direct}	supervision
RuleChef (22 rules)	.738	.719	.830	1,000 chunks
Longformer [†]	.836	.919	—	fine-tuned, 1,013 docs
RoBERTa NER [†]	.441	.906	—	off-the-shelf

Table 2: Official TAB test split (127 full documents), scored with the benchmark’s own evaluation script: token-level precision weighted by BERT information content, and mention-level recall over annotated spans (all identifiers / direct identifiers only). [†]: as reported by Pilán et al. (2022).

We also run the learned ruleset over the 127 full, unchunked documents of the official TAB test split and score it with the benchmark’s own evaluation script, which measures recall over annotated spans and token-level precision weighted by BERT-estimated information content. Table 2 compares results with the two baselines reported under the same protocol by Pilán et al. (2022). Performance of the 22 patterns learned by RuleChef remains well below that of the fine-tuned Longformer system on both precision and recall, but is already superior to the off-the-shelf RoBERTa system, achieving nearly 30 points higher precision and only 19 points lower recall. These results illustrate that RuleChef can generate fully transparent, inspectable and editable rule systems that are competitive with off-the-shelf neural models. In application scenarios where transparency, explainability, or auditability of decisions is critical, such high-precision systems can serve either as the basis of further development of high-performing rule systems or as the preferred model in a cascade of systems that also include models with higher recall at the cost of precision and/or explainability.

5.2 Ablation

A simple ablation study is performed to understand the impact of RuleChef’s iterative learning process on the quality of the final rule system. In particular this involves isolating the baseline performance of prompting an LLM to generate a rule system in a single pass. We observe that in such settings LLMs rely on the task description at least as much as on the initial training examples, and it is the subsequent iterations guided by additional examples and feedback on failures that allows LLMs to move from a generic rule system towards one that fits the actual task that is represented by the

Configuration	FMT F1	SEM F1	rules	calls
one-shot rule prompting	74.2	7.4	31	8
+ refinement on train	63.2	38.2	38	12
+ holdout acceptance	83.5	44.4	36	14
+ agentic coord. (3 iter.)	80.9	35.9	21	19
agentic, 8 iterations	81.7	47.8	9	33

Table 3: Ablation on TAB. Each row adds one component; “calls” counts all learning-time LLM calls (synthesis, patches, critic, audit). One-shot prompting writes plausible format regexes but is nearly useless on semantic types; the validated refinement loop does the work.

training dataset. Table 3 shows the effect of various refinement steps on the TAB dataset. Note that the row *+agentic coord.* corresponds to the main configuration evaluated in earlier sections, but the numbers differ (e.g. format F1 of 80.9 vs. 78.7) due to the non-determinism of RuleChef’s process that is inherent to the iterative prompting of LLMs (see also the section on Limitations).

Once again we observe a stark contrast between the two label groups *Format* (FMT) and *Semantic* (SEM). On the FMT group, which contains entity types easily identified by their format (dates, quantities, various identification numbers), a single pass of an LLM can create a ruleset that is within 5-10 percentage points of the best ones in terms of F-score. In case of the SEM group containing traditional NE types such as PER, LOC, and ORG an initial rule-system is practically useless.

Refining the rules by providing more training examples achieves substantial improvement on the SEM group, while the strong baseline on the FMT group deteriorates. However, the additional step of filtering new rules based on their performance on the dev set (*holdout acceptance*) substantially increases performance on both sets of entity types. The effect of *agentic coordination* varies greatly with the number of iterations. In this simple setup we see that 3 iterations lead to a decrease in performance compared to the strongest system on both groups, while 8 iterations produce a superior system on the SEM group but cannot restore the original top performance on the FMT group. The total number of rules across all entity types is of particular interest. After 3 iterations of the coordinator, the number of patterns is already reduced to 21 from the 36 of the previous system, and after 8 iterations it contains only 9 rules altogether, which still achieve top scores on the SEM group and are within 2 points of the best system on the FMT group.

We believe that these results, while preliminary,

already give a clear indication that a guided, iterative rule generation approach can result in robust rule-based systems even for challenging tasks like NER. Properly understanding the nature of how a rule system evolves as a result of such iterative improvement strategies and how this process varies across types of language processing tasks will require considerable further experimentation, including also the qualitative analysis of intermediate rule systems for each task.

5.3 Repairing Rules with Feedback

Because RuleChef models are lists of readable rules, its defects can also be addressed explicitly via human-in-the-loop (HITL) feedback. The next set of experiments measures the impact of this process. We inspected the learned rules and targeted three issues: the QUANTITY rule matched fraction-shaped case numbers like *1432/03*, CODE rule missed bare application numbers, and a PERSON rule for initials fired on ordinary abbreviations. We attached one sentence of feedback to each rule through the standard interface, e.g.:

“Never match number/number patterns like ‘1432/03’—those are case numbers, not quantities.”

Type	before F1	after F1	Δ
QUANTITY (feedback)	5.7	35.6	+29.8
CODE (feedback)	45.4	48.8	+3.4
PERSON (feedback)	53.4	54.0	+0.6
DATETIME (untouched)	85.1	85.1	0.0
DEM (untouched)	40.4	37.7	−2.7

Table 4: Results of a single round of human-in-the-loop repair using three sentences of rule-level feedback

Results are shown in Table 4. A single repair round (92 seconds, two LLM calls) substantially improves performance on the QUANTITY class (from 5.7 to 35.6 F1), while on the other two criticized classes the performance improves modestly. This workflow of improving individual rules based on a human’s understanding is one that hand-crafted rule systems always supported and neural models do not.

5.4 Qualitative Analysis of Learned Rules

Table 5 shows examples of rules learned on the TAB dataset, together with their precision and match count on the development set. Descriptive rule names have been generated by the LLMs. We

Rule name	Pattern (abridged)	dev P	<i>n</i>
single_date	\b\d{1,2}\s+(?:January ...)	.95	362
court_names	(?:District Regional Supreme ...)\s+Court	.79	118
titled_full_name	\b(?:Mr Mrs Dr Justice)\.?\s+[A-Z]...	.93	69
republic_kingdom_gov.	(?:Federal Republic of Germany ...)	.97	37
specific_institutions	(?:BBC Scotland House of Lords ...)	.47	30
case_and_echr_numbers	(?:no\.\s*)\d{4,6}/\d{2,4}	.86	22
initials_with_period	(?:[A-Z])\s{1,4}...	.35	17
lives_in_location	(?:lives resides)\s+in\s+[A-Z]...	1.00	12

Table 5: Examples of rules learned on the TAB dataset, ranked by the number of matches on the dev set.

observe that high-coverage high-precision rules encode task knowledge such as formatting rules (date formats in the rule `single_date`, honorifics in the rule `titled_full_name`, etc.) and lists of terms (`court_names`, `republic_kingdom_gov`). In contrast, the rule `lives_in_location` encodes patterns about the context in which entities of a certain type occur. We can also observe that some rules encoding lists of entities or formatting conventions achieve substantially lower precision than others. The list of organizations `specific_institutions` is overly generic, also containing standalone words such as *Parliament*, and so is the pattern `initials_with_period` that matches abbreviations that are not person names. These are the kinds of rules that must be the target of subsequent improvement steps that refine them based on examples of false positives presented to the LLM.

5.5 Few-Shot Classification and Observation Mode

We use the Banking77 dataset to test RuleChef on a simple 5-way intent classification task. The results in Table 6 show that simple LLM prompting as well as standard supervised learning approaches achieve precision and recall in the .95-.98 range, and that the rule system learned by RuleChef is competitive with these methods in terms of precision but not in terms of recall, a typical outcome for rule-based systems. The table also shows rule performance in various stages of RuleChef’s observation mode, where the system discovers labels and learns rules based on input-output examples from an external

	Rules	Cov.	Repl.	P	R	F1
LLM prompting	—	100	—	98.0	98.0	98.0
Zero-shot NLI (DeBERTa-v3)	—	100	—	96.5	96.5	96.5
LogReg (MiniLM emb.)	—	100	—	95.0	95.0	95.0
RuleChef (few-shot)	126	62.5	—	97.6	61.0	75.1
<i>Observation mode (no labels)</i>						
after 10 calls	14	20.5	19.5	92.7	19.0	31.5
after 25 calls	24	42.5	41.0	95.3	40.5	56.8
after 50 calls	40	49.5	48.0	96.0	49.5	63.6

Table 6: Results on the 200 dev set queries of the Banking77 dataset. Top rows compare rule performance to three baselines: simple LLM prompting, a logistic-regression classifier on MiniLM sentence embeddings, and the DeBERTa-v3 model MoritzLaurer/deberta-v3-base-zeroshot-v1.1-all-33 used for zero-shot classification, which scores each candidate label as an entailment hypothesis. Bottom rows show the performance of RuleChef’s *observation mode*, which discovers class labels and learns rules based on a growing number of input-output examples from the top-performing LLM-based approach. Coverage (Cov.) measures the fraction of inputs covered by any learned rule.

model (the top-performing LLM-based approach). Here we can see that as the number of observed examples grows from 10 to 50, precision grows from .93 to .96 and recall increases from .19 to .50. Regardless of whether or not our method would be able to further increase recall without sacrificing precision, a system such as the one learned based on only 50 examples and without human intervention is already robust enough to be used as the first tier of a hybrid system, replacing black box inference for half of the inputs, offering a clear gain in both transparency and computational cost.

6 Discussion and Future Work

We presented the RuleChef methodology for constructing rule-based systems for various NLP tasks via iterative refinement based on annotated examples and human feedback. Using a generic Named Entity Recognition task as our main testbed, we have shown that for classes of entities exhibiting highly regular surface patterns it is possible to learn fully transparent rule-based systems that are competitive with standard black box approaches. On the more challenging groups of entities RuleChef can produce a set of high-precision rules with substantial recall that can form the basis of subsequent improvements and can function as the preferred model in hybrid systems, increasing explainability and reducing computational cost.

Our simple ablation study offers a preliminary

evaluation of the various strategies for iterative learning implemented by RuleChef. In addition to the process of introducing training examples gradually and only accepting refinements that improve overall performance, we also showcase the capabilities of the *agentic coordination* feature, which allows an external model to orchestrate learning steps based on observed rule performance and perceived rule quality. RuleChef is released as open-source software under an Apache 2.0 license², including all code necessary to reproduce the experimental results presented in this paper.

RuleChef is a generic framework implementing a variety of approaches to rule learning for NLP, and the empirical results presented in this paper are strictly preliminary. We expect that the utility of each RuleChef feature will vary greatly across domains, genres, and datasets. The main approaches described in Section 3 would each benefit from in-depth experimental evaluation. The quality of rule synthesis via LLMs depends on the choice of prompts and their ability to separate training signals from pre-existing model bias. Iterative improvement by introducing additional training examples depends heavily on chunking and sampling strategies as well as on the acceptance criteria for newly introduced rules. The use of direct human feedback should be explored under various constraints on the form and content of such user input, while agentic coordination depends on stand-alone models for assessing and criticizing rules, a complex task in its own right.

Limitations

The broad range of methods introduced and evaluated in this paper introduce a number of limitations. Quantitative results presented are based on single runs and fixed splits. Repeated runs of selected configurations show variations in F1 performance of approx. ± 3 points even for the more regular FORMAT group of entities, and we report metrics from representative experiments rather than means over multiple runs. All experiments use English texts as input, and a single LLM family for rule generation (Kimi-K2). Each main method involving LLM invocations also introduces the need for the systematic evaluation of how model bias influences the rule learning process and how this may introduce bias into the final rule systems.

²<https://github.com/KRLabsOrg/rulechef>

Acknowledgments

Work partially supported by the “CLEAR” project, funded within the Cybersecurity Programme Kybernet-Pass of the Austrian Federal Ministry of Finance and managed by the Austrian Research Promotion Agency.

References

- Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. [Language models enable simple systems for generating structured views of heterogeneous data lakes](#). *Proceedings of the VLDB Endowment*, 17(2):92–105.
- Laura Banarescu, Claire Bonial, Shu Cai, Madalina Georgescu, Kira Griffitt, Ulf Hermjakob, Kevin Knight, Philipp Koehn, Martha Palmer, and Nathan Schneider. 2013. [Abstract meaning representation for sembanking](#). In *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*, pages 178–186. ACL.
- Alberto Bartoli, Andrea De Lorenzo, Eric Medvet, and Fabiano Tarlao. 2016. Inference of regular expressions for text extraction from examples. *IEEE Transactions on Knowledge and Data Engineering*, 28(5):1217–1230.
- Alberto Bartoli, Andrea De Lorenzo, Eric Medvet, and Fabiano Tarlao. 2018. Active learning of regular expressions for entity extraction. *IEEE Transactions on Cybernetics*, 48(3):1067–1080.
- Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. 2020. [Efficient intent detection with dual sentence encoders](#). In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*, pages 38–45. ACL.
- Qiaochu Chen, Xinyu Wang, Xi Ye, Greg Durrett, and Isil Dillig. 2020. Multi-modal synthesis of regular expressions. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 487–502. ACM.
- Laura Chiticariu, Yunyao Li, and Frederick R. Reiss. 2013. Rule-based information extraction is dead! long live rule-based information extraction systems! In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 827–832. ACL.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: Pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186. ACL.

- Kaiyu He and Zhiyu Chen. 2025. [From reasoning to learning: A survey on hypothesis discovery and rule learning with large language models](#). *Transactions on Machine Learning Research (TMLR)*. ArXiv:2505.21935.
- Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8003–8017. ACL.
- Ádám Kovács, Kinga Gémes, Eszter Iklódi, and Gábor Recski. 2022. [POTATO: exPlainable infOrmation exTrAcTiOn framework](#). In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM)*, pages 4897–4901. ACM.
- Piyawat Lertvittayakumjorn, Leshem Choshen, Eyal Shnarch, and Francesca Toni. 2022. GrASP: A library for extracting and exploring human-interpretable textual patterns. In *Proceedings of the 13th Language Resources and Evaluation Conference (LREC)*, pages 6093–6103. ELRA.
- Yeting Li, Shuaimin Li, Zhiwu Xu, Jialun Cao, Zixuan Chen, Yun Hu, Haiming Chen, and Shing-Chi Cheung. 2021. TransRegex: Multi-modal regular expression synthesis by generate-and-repair. In *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*, pages 1210–1222. IEEE.
- Zachary C. Lipton. 2018. [The mythos of model interpretability](#). *Queue*, 16(3):31–57.
- Nicholas Locascio, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman, and Regina Barzilay. 2016. Neural generation of regular expressions from natural language with minimal domain knowledge. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1918–1923. ACL.
- Joakim Nivre, Marie-Catherine de Marneffe, Filip Ginter, Jan Hajič, Christopher D. Manning, Sampo Pyysalo, Sebastian Schuster, Francis Tyers, and Daniel Zeman. 2020. [Universal Dependencies v2: An evergrowing multilingual treebank collection](#). In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 4034–4043, Marseille, France. European Language Resources Association.
- Ildikó Pilán, Pierre Lison, Lilja Øvrelid, Anthi Papadopoulou, David Sánchez, and Montserrat Batet. 2022. The text anonymization benchmark (TAB): A dedicated corpus and evaluation framework for text anonymization. *Computational Linguistics*, 48(4):1053–1101.
- Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282.
- Prithviraj Sen, Yunyao Li, Eser Kandogan, Yiwei Yang, and Walter Lasecki. 2019. HEIDL: Learning linguistic expressions with deep learning and human-in-the-loop. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations (ACL)*, pages 135–140. ACL.
- Eyal Shnarch, Ran Levy, Vikas Raykar, and Noam Slonim. 2017. GrASP: Rich patterns for argumentation mining. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1345–1350. ACL.
- Shangeetha Sivasothy, Scott Barnett, Rena Logothetis, Mohamed Abdelrazek, Zafaryab Rasool, Srikanth Thudumu, and Zac Brannelly. 2024. [Large language models for generating rules, yay or nay?](#) *arXiv preprint arXiv:2406.06835*.
- Ryan Smith, Jason A. Fries, Braden Hancock, and Stephen H. Bach. 2024. [Language models in the loop: Incorporating prompting into weak supervision](#). *ACM/IMS Journal of Data Science*, 1(2):1–30.
- Peter M. Stahl. 2019. [grex – a command-line tool and rust library for generating regular expressions from user-provided test cases](#).
- Marco A. Valenzuela-Escárcega, Gus Hahn-Powell, and Dane Bell. 2020. Odinson: A fast rule-based information extraction framework. In *Proceedings of the 12th Language Resources and Evaluation Conference (LREC)*, pages 2183–2191. ELRA.
- Paroma Varma and Christopher Ré. 2018. Snuba: Automating weak supervision to label training data. *Proceedings of the VLDB Endowment*, 12(3):223–236.
- Ruo Cheng Wang, Eric Zelikman, Gabriel Poesia, Yewen Pu, Nick Haber, and Noah D. Goodman. 2024. Hypothesis search: Inductive reasoning with language models. In *The Twelfth International Conference on Learning Representations (ICLR)*. ArXiv:2309.05660.
- Peter West, Chandra Bhagavatula, Jack Hessel, Jena D. Hwang, Liwei Jiang, Ronan Le Bras, Ximing Lu, Sean Welleck, and Yejin Choi. 2022. [Symbolic knowledge distillation: from general language models to commonsense models](#). In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4602–4625. ACL.
- Edwin B. Wilson. 1927. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212.
- Peilin Yu and Stephen H. Bach. 2023. Alfred: A system for prompted weak supervision. In *Proceedings of the 61st Annual Meeting of the Association for*

Computational Linguistics: System Demonstrations (ACL), pages 479–488. ACL.

Urchade Zaratiana, Gil Pasternak, Oliver Boyd, George Hurn-Maloney, and Ash Lewis. 2025. [GLiNER2: Schema-driven multi-task learning for structured information extraction](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 130–140. ACL.

Urchade Zaratiana, Nadi Tomeh, Pierre Holat, and Thierry Charnois. 2024. GLiNER: Generalist model for named entity recognition using bidirectional transformer. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 5364–5376. ACL.

Jianguo Zhang, Trung Bui, Seunghyun Yoon, Xiang Chen, Zhiwei Liu, Congying Xia, Quan Hung Tran, Walter Chang, and Philip Yu. 2021. Few-shot intent detection via contrastive pre-training and fine-tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1906–1912.

Zexuan Zhong, Jiaqi Guo, Wei Yang, Jian Peng, Tao Xie, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. 2018. SemRegex: A semantics-based approach for generating regular expressions from natural language specifications. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1608–1618. ACL.

Wenxuan Zhou, Sheng Zhang, Yu Gu, Muhao Chen, and Hoifung Poon. 2024. [UniversalNER: Targeted distillation from large language models for open named entity recognition](#). In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. ArXiv:2308.03279.

A Prompt Templates

RuleChef uses large language models exclusively at learning time, issuing calls through eleven distinct prompt templates that span every phase of rule synthesis, refinement, agentic coordination, and observation. The prompts fall into four functional groups.

A.1 Learning prompts

The *synthesis prompt* (Figure 3) is the primary learning call: it is issued once at the start of a run to generate an initial rule set from the full training dataset. For multi-class tasks, the *per-class synthesis prompt* (Figure 4) is used instead, producing one focused call per label with its positive examples and counter-examples from other classes, preventing cross-class interference. The *patch prompt* (Figure 5) is issued once per refinement iteration

to incrementally update the rule set by fixing observed failures; it receives the current rule set with per-rule metrics alongside a clustered, sampled set of errors. Six variants of the patch prompt (from richest to most compact) are pre-built; RuleChef selects the longest one fitting the context window.

A.2 Agentic coordinator prompts

When the agentic coordinator is enabled, four additional prompts drive autonomous decision-making. After each refinement iteration the *refinement coordinator prompt* (Figure 6) analyses per-class metrics and returns strategic guidance for the next patch call. Periodically, the *rule critic prompt* (Figure 8) performs a holistic review of the rule set and produces actionable per-rule feedback, which re-enters the system through the standard feedback interface and is visible in subsequent patch prompts. The *rule auditor prompt* (Figure 7) compares rules for overlap or redundancy and proposes merge, remove, or tighten actions; any structural change is verified on the held-out split before being accepted. When RuleChef is configured to learn from a live data stream, the *learning trigger decision prompt* (Figure 9) decides whether accumulated new examples warrant starting a retraining loop.

A.3 Observation mode prompts

In observation mode, where RuleChef bootstraps rules from an existing LLM’s input-output pairs rather than from manually labelled data, two prompts are used. The *task discovery prompt* (Figure 10) infers the task schema (type, input/output fields) from a sample of raw API call logs. The *observation mapping prompt* (Figure 11) then maps each batch of up to ten raw observations to the discovered schema, filtering irrelevant calls.

A.4 Utility prompts

Two lightweight prompts cover auxiliary functions. The *synthetic example generation prompt* (Figure 12) generates a single realistic training input on demand, used to augment sparse training sets. The *LLM fallback prompt* (Figure 13) is the only prompt active at inference time: it is issued as a last resort when no rule fires and the executor is configured to delegate to an LLM rather than abstain.

```

Task: {task.name}
Description: {task.description}

Input schema: {task.input_schema}
Output schema:
{task.output_schema}
[AVAILABLE ENTITY TYPES: {labels}          ## only for NER / classification tasks
Rules MUST use one of these types in output_template.]

CORRECTIONS (Learn from failures - {n} shown):  ## omitted when dataset has no corrections

Input: {json(correction.input)}
Got (WRONG): {json(correction.model_output)}
Expected (CORRECT): {json(correction.expected_output)}
[Feedback: {correction.feedback}]          ## included only when per-correction feedback is set

TRAINING EXAMPLES ({n} shown):

Input: {json(example.input)}
Output: {json(example.expected_output)}

{data_evidence}  ## task-type-specific section: entity counts, regex-derived regex hints,
                ## class distributions; omitted when evidence is empty

[USER FEEDBACK:          ## optional; omitted when no feedback is attached
- {feedback_item}]

[EXISTING RULES ({n} current):          ## optional; included on re-synthesis when rules exist
- {rule.name} (priority {rule.priority}, success: {success_rate})
  Format: {rule.format}
  Pattern: {rule.content[:100]}...
  Confidence: {rule.confidence}
CONSIDER:
- Refine existing high-performing rules
- Fix or replace low-performing rules
- Keep rules that work well
- Add new rules for uncovered patterns]

YOUR TASK:
(Update and refine | Synthesize a complete) ruleset (max {max_rules} rules) that:
1. Handles all corrections correctly (CRITICAL - these show failure modes)
2. Works on all examples
3. Respects user feedback
4. Is general and minimal (avoid redundant rules)

WHAT MAKES A GOOD RULE:
- PRECISION OVER RECALL: A rule that matches 10 things correctly beats one that matches 100 with 20 wrong. Never sacrifice precision for recall.
  Missing a match is fixable later; a wrong match poisons results.
- GENERALIZE, DON'T MEMORIZE: Rules run on unseen text. Match the *structure* of values, not the exact strings you see in training data.
- USE CONTEXT: The same string can mean different things. Match surrounding words/punctuation and use capture groups ($1) to extract just the value.
- ONE RULE, ONE PATTERN: Multiple focused rules beat one giant regex. Each rule should have a clear, testable reason to exist.
- THINK ADVERSARIALLY: Before writing a rule, ask "what else could this match?" If the answer isn't empty, narrow the pattern or add context.

RULES CAN BE:
[- Regex patterns (for structured extraction)]  ## each line included only if the format is allowed
[- Python code (for complex logic)]
[- spaCy token matcher patterns (for linguistic/NLP patterns)]

IMPORTANT: You must ONLY use the allowed formats listed above.
[IMPORTANT: For CODE rules, write standard multi-line Python functions with proper indentation.]
[IMPORTANT: spaCy rules must be valid JSON arrays of token dicts (spaCy Matcher format).]
[IMPORTANT: spaCy NER is disabled. Do NOT use ENT_TYPE or ENT_ID in spaCy patterns.]

[REGEX SYNTAX REFERENCE:          ## included when REGEX format is allowed
- \b word boundaries, (?...) non-capturing groups, (a|b|c) alternation
- [A-Z] [a-z] \d \s character classes, +*{n,m} quantifiers
- (?=...) lookahead, (?<...) lookbehind -- match context without consuming it CAPTURE GROUPS ($1) vs FULL MATCH ($0): Use $0 when the entire match IS
  the value; use $1 when context is needed. With "text": "$1", $start/$end auto-adjust to the group position.
GOOD vs BAD PATTERNS:
BAD: "\d+" -- matches every number; most will be wrong type
GOOD: "(d+)\s*(?:kg|lbs|oz)" -- only numbers followed by weight units
BAD: "Alice|Bob|Tokyo" -- memorizes training strings, misses unseen values
BAD: "[A-Z][a-z]+" -- matches any capitalized word (too broad)
GOOD: "(?:in|from|at|near)\s+([A-Z][a-z]+(?:\s+[A-Z][a-z]+)*)" -- locations after prepositions
Prefer structural patterns over memorized alternations.]

[SPACY TECHNIQUES:          ## included when SPACY format is allowed
- Use POS tags (PROPN, NOUN, VERB), DEP labels (nsubj, dobj, pobj), SHAPE for structural patterns.
- Use IN for value sets; OP for quantifiers ("?" optional, "+" one or more).]

(...) ## instructions on return format omitted due to space constraints

[{code_or_spacy_format_examples}]          ## task-type-specific format examples, when applicable

Focus on learning from CORRECTIONS - they show exactly what went wrong!

IMPORTANT: Return ONLY valid JSON. Ensure:
- All strings use double quotes and are properly escaped
- All braces and brackets are balanced
- No trailing commas
- Response is complete (not truncated)

```

Figure 3: Rule synthesis prompt for initial rule generation. Square brackets mark optional sections; {name} marks variable slots. Lines beginning with ## are explanatory annotations, not prompt content. The data evidence section contains task-specific pattern statistics and regex-derived regex hints.

```

Task: {task.name}
Description: {task.description}

Input schema: {task.input_schema}
Output schema:
{task.output_schema}
[AVAILABLE ENTITY TYPES: {labels}
Rules MUST use one of these types in output_template.]

FOCUS: Generate rules for class '{target_class}' ONLY.
## (or: "to extract the '{target_class}' field" for TRANSFORMATION tasks)

POSITIVE EXAMPLES for '{target_class}' ({n} total):

Input: {json(example.input)}
Output: {json(example.expected_output)}

[COUNTER-EXAMPLES (these are NOT '{target_class}' -- your rules must NOT match these):
## omitted for TRANSFORMATION tasks
Input: {json(counter_example.input)}
Label: {json(counter_example.expected_output)}]

{data_evidence}

{format_instructions}

WHAT MAKES A GOOD RULE: ## RULE_QUALITY_GUIDE constant (full text in Figure 3)
[...]

INSTRUCTIONS:
- Generate up to {max_rules} rules that match examples of '{target_class}'.
- Rules should generalize to unseen text, not just memorize the examples shown.
- Use structural patterns with word boundaries that generalize to unseen text.
[- Rules must NOT match the counter-examples shown above.] ## omitted for TRANSFORMATION

{response_schema}

{format_examples}

Focus on learning from CORRECTIONS - they show exactly what went wrong!

IMPORTANT: Return ONLY valid JSON. Ensure:
- All strings use double quotes and are properly escaped
- All braces and brackets are balanced
- No trailing commas
- Response is complete (not truncated)

```

Figure 4: Per-class synthesis prompt. Used in place of the synthesis prompt for multi-class tasks: one call is issued per class label, showing only that class's positive examples and counter-examples from other classes. Prevents cross-class interference in the generated rules. Not used for binary or untyped extraction tasks.


```

You are updating an existing rule-based extractor. Do NOT rewrite good rules; add or adjust
only what is needed.

Task: {task.name}
Description: {task.description}

Input schema: {task.input_schema}
Output schema: {task.output_schema}
[AVAILABLE ENTITY TYPES: {labels}]
Rules MUST use one of these types in output_template.]

{data_evidence}                                ## omitted in compact variants to save tokens

[USER GUIDANCE (task-level feedback):
- {feedback_item}]

[COORDINATOR GUIDANCE (prioritize this):
{guidance}]                                     ## injected by AgenticCoordinator.guide_refinement()

[PER-CLASS METRICS (current performance):
{class}: P={p} R={r} F1={f1} (TP={tp} FP={fp} FN={fn})]

CURRENT RULES (full details, note any user_feedback on specific rules):
{json(rules_detail)}                            ## each entry: name, format, pattern, metrics, user_feedback

[OTHER CURRENT RULES (compact index; avoid duplicating these names/labels):
{json(compact_rules)}]                         ## non-relevant rules shown compact in the "relevant" variant

{failure_summary}                             ## mode-based count of all failures, not just the sampled set below
FAILURES TO FIX (sampled, corrections are high priority):
{json(failure_snippets)}                       ## up to 20/10/5 items depending on variant; corrections: is_correction=true

[FALSE POSITIVES (rules are incorrectly matching these -- tighten the responsible rules):
  Predicted "{predicted_text}" as {predicted_type}
  -- {not an entity | should be {correct_type}}]

Instructions:
- Add, tweak, or DELETE rules to fix the shown failures and reduce false positives.
- Pay close attention to user_feedback on rules AND task-level USER GUIDANCE -- these are direct
  instructions from the user and MUST be addressed even if there are no failures.
- If a rule has user_feedback, modify or replace that rule to address the feedback.
- IMPORTANT: When updating an existing rule, you MUST reuse the EXACT same "name" as the original.
  Do NOT add suffixes like "_fixed", "_v2", "_updated", etc. The merge system uses name-matching to
  replace the old version -- a different name creates a duplicate instead of replacing.
- If a rule is fundamentally too broad (FP >> TP) and you're providing better, narrower
  replacements, list the old rule's exact name in "deleted_rules". Only delete if you're
  providing replacements in "rules".
- If a rule has high false positives, TIGHTEN its pattern or DELETE it and add narrower
  replacements. Adding context or narrowing the match is better than piling on new rules.
- Use structural patterns with word boundaries that generalize to unseen text.
- Keep total new/updated rules <= {max_rules}.
- Use formats: {allowed_formats}
- Avoid touching unrelated behaviors.

WHAT MAKES A GOOD RULE:                        ## RULE_QUALITY_GUIDE constant (full text in Figure 3)
[...]

{format_instructions}

{response_schema}

```

Figure 5: Patch prompt. Issued once per refinement iteration to incrementally fix failures. Six variants are pre-built and differ in the number of failure snippets shown (20, 10, or 5), whether the data evidence section is included, and whether non-relevant rules are shown in compact or full form. RuleChef selects the longest variant that fits the context window. Lines beginning with ## are annotations in this figure only.

```

You are the Refinement Coordinator for a rule learning
system. After each refinement iteration, you analyze
per-class performance and guide the next patch.

ITERATION: {iteration + 1}/{max_iterations}
OVERALL: accuracy={exact_match}, micro_F1={micro_f1},
        macro_F1={macro_f1}

PER-CLASS PERFORMANCE (sorted worst to best):
{class}: F1={f1} P={p} R={r} (TP={tp} FP={fp} FN={fn})
[...]

Return JSON:
{
  "focus_classes": ["class names needing most improvement"],
  "guidance": "Specific advice for the rule generator --
              which classes to prioritize, what patterns to try,
              what to avoid. 2-3 sentences max.",
  "should_continue": boolean
}

```

Figure 6: Refinement coordinator prompt. Issued after each iteration when the agentic coordinator is enabled. The returned guidance string is injected verbatim into the next patch prompt as coordinator guidance; `should_continue=false` terminates the refinement loop early.

```

You are a Rule Auditor for a rule-based extraction/classification system.

Analyze these {n} rules and their per-rule metrics.

RULES:
{json(rule_entries)} ## each entry: id, name, format, pattern[:300], priority,
                        ## output_key, output_template, precision, recall, F1, TP, FP

Your job is to CONSOLIDATE and CLEAN the ruleset.

ACTIONS (in priority order):
1. MERGE: Two+ rules with similar/overlapping patterns targeting the same output/label.
   Combine into one rule. Only merge rules of the same format and same output_template/output_key.
2. REMOVE rules that hurt more than they help:
   - precision=0 AND matches>0 (pure noise -- every match is wrong)
   - false_positives > 2x true_positives (rule causes more harm than good)
   - Memorized exact strings from training data that won't generalize
3. TIGHTEN: If a rule has high FP, return it as a merge-with-self -- same rule_id but narrower
   pattern.

IMPORTANT -- do NOT remove:
- The only rule for a class/label -- even if it looks weak, tighten it instead
- Rules with 0 matches -- the training set may be small, they could help on unseen data

LOOK FOR:
- Near-duplicate rules (same type, similar regex) -- merge them
- Overly broad patterns (like bare \d+ or [A-Z][a-z]+) with high FP -- tighten or remove
- Rules that are subsets of other rules (one pattern already covered by another)

Return JSON:
{
  "analysis": "Brief summary (1-2 sentences)",
  "actions": [
    {"action": "merge", "rule_ids": ["id1", "id2"], "merged_pattern": "new regex",
     "merged_name": "Combined rule name", "reason": "why"},
    {"action": "remove", "rule_ids": ["id"], "reason": "why"}
  ]
}

Return {"analysis": "All rules are useful", "actions": []} if no changes needed.

```

Figure 7: Rule auditor prompt. Issued periodically by the agentic coordinator to merge redundant rules, remove pure-noise rules, and tighten over-broad patterns. Any structural change proposed here is applied and then re-evaluated on the held-out split; changes that lower F1 are reverted. Only fires when there are at least two rules.

You are an expert Rule Critic acting as a human domain expert. You are reviewing a rule-based {task.type} system and providing actionable feedback.

TASK: {task.name}
{task.description}
Type: {task.type}
Input schema: {json(task.input_schema)}
Output schema: {json(task.output_schema)}

OVERALL PERFORMANCE:
micro F1={micro_f1}, P={micro_precision}, R={micro_recall}
exact_match={exact_match} ({total_docs} documents)

PER-CLASS PERFORMANCE (sorted worst-first):
{class}: P={p} R={r} F1={f1} (TP={tp} FP={fp} FN={fn})

RULES ({n} total):
Rule: "{rule.name}" (id={rule.id})
Format: {rule.format}, Priority: {rule.priority}
Pattern: {rule.content}
[Output template: {json(rule.output_template)}]
[Output key: {rule.output_key}]
Metrics: P={p} R={r} F1={f1} (TP={tp} FP={fp}, {total} total matches)
[FP examples from this rule:
Input: "{input_context[:150]}"
Rule produced: {json(rule_output[:3])}
Expected: {json(expected[:3])}]

[FALSE POSITIVES (system-level, {n} examples):
Predicted "{predicted_text}" as {predicted_type}
-- {not an entity | should be {correct_type}} (context: "{context[:80]}")]

[MISSED ENTITIES (sample documents with errors):
Input: "{input_text[:150]}"
Expected: {json(expected)}
Got: {json(got)}]

ANALYZE HOLISTICALLY:
1. Which rules cause the most harm and WHY? Show your reasoning.
2. Are there inter-class conflicts? (same text matched by rules for different types)
3. Are priority assignments correct? (higher priority runs first, wins conflicts)
4. What patterns are MISSING for classes with low recall?
5. What would a human regex expert change about these patterns?

PROVIDE FEEDBACK:
- rule_feedback: For EACH problematic rule, provide SPECIFIC, ACTIONABLE advice.
Bad: "This rule is too broad" (vague)
Good: "Narrow \d+ by adding word-boundary context: use (\d+)\s*(?:million|billion) for large numbers, and let MONEY/PERCENT rules handle \$-prefixed numbers by higher priority"
- task_guidance: Strategic advice about the ENTIRE ruleset.

Return JSON:
{
 "analysis": "1-2 sentence summary of the main issues",
 "rule_feedback": {
 "{rule_id}": "Specific actionable advice..."
 },
 "task_guidance": "Strategic guidance about the full ruleset..."
}

Figure 8: Rule critic prompt. Issued periodically by the agentic coordinator to perform holistic expert review of the full rule set. The returned rule_feedback entries are attached to the named rules as feedback items; task_guidance is stored as task-level feedback. Both re-enter the system through the same interface as human feedback and are visible in subsequent patch prompts.


```

You are the Coordinator for a rule learning system.
Decide if we should trigger a retraining loop NOW
based on new data.

STATUS:
- New examples: {n_examples}
- New corrections (high priority): {n_corrections}
- Current rules: {n_rules}

NEW DATA SAMPLES (up to 10):
- [CORRECTION] Input: {json(ex.input)} -> Output: {json(ex.output)}
- [EXAMPLE] Input: {json(ex.input)} -> Output: {json(ex.output)}

DECISION CRITERIA:
1. TRIGGER if we have corrections (users fixing mistakes).
2. TRIGGER if we have a significant batch of new examples (5+).
3. WAIT if data looks sparse or redundant.

STRATEGIES:
- 'balanced': Standard mix (default)
- 'corrections_first': If we have corrections
- 'diversity': If we have many similar examples
- 'uncertain': If examples look ambiguous

Return JSON:
{
  "should_learn": boolean,
  "strategy": "balanced"|"corrections_first"|"diversity"|"uncertain",
  "max_iterations": integer (1-3),
  "reasoning": "Short explanation"
}

```

Figure 9: Learning trigger decision prompt. Issued when RuleChef operates in streaming or batch-wise mode and a new buffer of data has accumulated. The coordinator decides whether to start a retraining loop immediately or wait for more data, and selects the training strategy and iteration budget.

```

You are analyzing LLM API calls to discover the
underlying task pattern.

Here are {n} sample LLM interactions:

{obs_text}    ## formatted raw API logs:
               ## system prompt, user message, assistant
               response

Based on these interactions, identify:
1. What task is being performed? (name and description)
2. What type of task? Choose ONE: extraction, ner,
   classification, transformation
   - extraction: finding text spans (untyped)
   - ner: finding typed entities with labels
   - classification: assigning a label to input text
   - transformation: extracting structured fields from text
3. What are the input fields and their types?
4. What are the output fields and their types?
5. Which input field contains the main text?

Return ONLY valid JSON:
{
  "name": "task_name",
  "description": "one sentence description",
  "type": "classification",
  "input_schema": {"text": "str"},
  "output_schema": {"label": "str"},
  "text_field": "text"
}

```

Figure 10: Task discovery prompt. Issued once in observation mode when no task schema is provided upfront. Requires a minimum of accumulated raw observations (default: 5) before firing. The discovered schema is used for all subsequent observation mapping calls.

```

You are extracting structured training data from LLM
interactions.

Task definition:
Name: {task.name}
Description: {task.description}
Type: {task.type}
Input schema: {json(task.input_schema)}
Output schema: {json(task.output_schema)}

Here are {n} LLM interactions to analyze:

{obs_text}    ## up to 10 raw API call logs per batch

For EACH interaction, determine:
1. Is it relevant to the task above? (relevant: true/false)
2. If relevant, extract the input (matching input_schema
   keys) and output (matching output_schema keys).

Return ONLY a JSON array with exactly {n} objects:
[
  {"relevant": true, "input": {...}, "output": {...}},
  {"relevant": false, "input": null, "output": null},
  ...
]

```

Figure 11: Observation mapping prompt. Issued once per batch of up to ten raw API call logs. Filters irrelevant calls and extracts structured input-output pairs matching the task schema. Mapped examples are added to the training buffer and consumed by subsequent synthesis or patch calls.

```
Generate a realistic training example for this task:  
  
Task: {task.name}  
Description: {task.description}  
Input schema: {task.input_schema}  
  
Return JSON with input fields only.  
Example #{seed + 1}:
```

Figure 12: Synthetic example generation prompt, generates a single realistic training input on demand. The seed parameter drives diversity across calls.

```
Task: {task.name}  
Description: {task.description}  
  
Input: {json(input_data)}  
  
Output schema: {task.output_schema}  
  
Return ONLY valid JSON matching the output schema,  
no explanation.
```

Figure 13: LLM fallback execution prompt: the only prompt active at inference time, issued only when no rule fires and the executor is not configured to abstain.